ОТЧЁТ

по лабораторной работе №4

«Модули, исключения, файлы и сериализация данных в Python»

Вариант 20. Логистика

1. Цель работы

Получить практические навыки разработки многофайлового Python-приложения, обработки исключительных ситуаций и сохранения данных во внешних файлах. В работе осваиваются модули и импорт, собственные исключения, конструкция try/except/else/finally, запись и чтение JSON и CSV, сериализация объектов и восстановление состояния программы после повторного запуска.

2. Постановка индивидуального задания

Вариант 20. Логистика. Нужно создать систему управления грузами. Для каждого груза хранятся номер, отправитель, получатель, масса и статус. Программа должна создавать груз, менять статус, искать и фильтровать записи, считать статистику и сохранять данные в JSON. Попытка изменить несуществующий груз должна приводить к собственному исключению.

По общим требованиям методички проект разделён на модули models.py, services.py, storage.py, exceptions.py и main.py. Объект переводится в словарь и собирается обратно через classmethod from_dict. Дополнительно сделаны экспорт и импорт CSV, резервная копия JSON и журнал операций.

3. Краткие теоретические сведения

Модуль в Python — это файл с расширением .py. Команда import подключает модуль целиком, а from module import name берёт из него конкретное имя. Такое разделение не даёт смешивать модель, бизнес-логику, хранение и меню в одном файле.

Исключение — это сигнал об ошибке во время выполнения. Блок try содержит опасный код, except обрабатывает выбранный тип ошибки, else выполняется, если ошибки не было, finally выполняется всегда. Своё исключение создаётся классом, который наследует Exception. Если исключение не поймать, программа завершится аварийно.

JSON хранит списки и словари и удобен для обмена данными. Модуль json записывает объект функцией dump и читает его функцией load. Обычный экземпляр класса в JSON сам не превращается, поэтому у Cargo есть to_dict и

from_dict. CSV — текстовая таблица; DictReader читает строку как словарь. Конструкция with open(...) закрывает файл даже при ошибке.

4. Описание программы

Класс Cargo проверяет номер, отправителя, получателя, положительную массу и статус из фиксированного набора. CargoService добавляет груз, ищет его, меняет статус и считает статистику. Изменение неизвестного номера поднимает CargoNotFoundError. Отменённый и уже доставленный груз нельзя перевести в недопустимое состояние.

JsonRepository загружает список при старте. Если файла ещё нет, программа не падает и начинает с пустого списка. При сохранении предыдущий JSON копируется в каталог backup. Экспорт CSV пишет те же поля, импорт собирает объекты через from_dict. Каждая операция дописывается в operations.log.

Меню работает в цикле, пока не выбран пункт 0. Нечисловая масса, нулевая масса, пустая строка и неизвестный пункт меню сообщаются пользователю без аварийного завершения.

5. Исходный код программы

Ниже приведён исходный код разработанных файлов.

Листинг — main.py

```
"""
Лабораторная работа №4. Вариант 20.
Система управления грузами: модули, исключения, JSON и CSV.
"""


import os
import sys


from exceptions import CargoError, CargoNotFoundError, StorageError, ValidationError
from models import STATUSES
from services import CargoService
from storage import JsonRepository


if sys.platform == "win32":
    sys.stdout.reconfigure(encoding="utf-8")
    sys.stderr.reconfigure(encoding="utf-8")
    if not sys.stdin.isatty():
        sys.stdin.reconfigure(encoding="utf-8")


BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(BASE_DIR, "data")



def read_line(prompt):
    if sys.stdin.isatty():
        return input(prompt).strip()
    raw = sys.stdin.readline()
    if raw == "":
        raise EOFError
    text = raw.rstrip("\r\n")
```

```
    print(f"{prompt}{text}")
    return text.strip()


def read_non_empty(prompt):
    while True:
        text = read_line(prompt)
        if text:
            return text
        print("Ошибка: значение не должно быть пустым.")


def read_positive_float(prompt):
    while True:
        text = read_line(prompt).replace(",", ".", 1)
        try:
            value = float(text)
        except ValueError:
            print("Ошибка: необходимо ввести число.")
            continue
        if value <= 0:
            print("Ошибка: масса должна быть больше нуля.")
            continue
        return value


def print_cargos(cargos, title="Список грузов"):
    print(f"\n{title}:")
    if not cargos:
        print("Подходящих грузов нет.")
        return
    print(f"{'N':<4}{'Номер':<12}{'Отправитель':<18}{'Получатель':<18}{'Масса':<10}{'Статус'}")
    for index, cargo in enumerate(cargos, start=1):
        print(
            f"{index:<4}{cargo.number:<12}{cargo.sender:<18}"
            f"{cargo.recipient:<18}{cargo.mass:<10.1f}{cargo.status}"
        )


def read_status():
    print("Статус:")
    for index, status in enumerate(STATUSES, start=1):
        print(f"  {index}. {status}")
    while True:
        text = read_line("Выберите статус (1-4): ")
        if text.isdigit() and 1 <= int(text) <= len(STATUSES):
            return STATUSES[int(text) - 1]
        print("Ошибка: выберите номер статуса от 1 до 4.")


def create_cargo(service):
    try:
        number = read_non_empty("  Номер груза: ")
        sender = read_non_empty("  Отправитель: ")
        recipient = read_non_empty("  Получатель: ")
        mass = read_positive_float("  Масса, кг: ")
        status = read_status()
        cargo = service.add(number, sender, recipient, mass, status)
    except CargoError as exc:
        print(f"Ошибка: {exc}")
        return
    else:
        print("Груз создан.")
        print(cargo)
    finally:
        print("Операция создания завершена.")


def change_status(service):
    number = read_non_empty("Номер груза: ")
    status = read_status()
    try:
        cargo = service.change_status(number, status)
```

```
    except CargoNotFoundError as exc:
        print(f"Исключение CargoNotFoundError: {exc}")
    except ValidationError as exc:
        print(f"Ошибка: {exc}")
    else:
        print("Статус изменён.")
        print(cargo)


def search_cargo(service):
    text = read_non_empty("Фрагмент номера, отправителя или получателя: ")
    print_cargos(service.search(text), f"Поиск «{text}»")


def filter_cargos(service):
    print("1. По статусу")
    print("2. По массе")
    choice = read_line("Выберите фильтр: ")
    if choice == "1":
        status = read_status()
        print_cargos(service.filter_by_status(status), f"Статус «{status}»")
    elif choice == "2":
        min_mass = read_positive_float("Минимальная масса, кг: ")
        max_mass = read_positive_float("Максимальная масса, кг: ")
        if max_mass < min_mass:
            print("Ошибка: максимум не может быть меньше минимума.")
            return
        print_cargos(
            service.filter_by_mass(min_mass, max_mass),
            f"Масса от {min_mass:.1f} до {max_mass:.1f} кг",
        )
    else:
        print("Ошибка: выберите 1 или 2.")


def show_statistics(service):
    stats = service.statistics()
    print("\n----- Статистика -----")
    print(f"Количество грузов: {stats['count']}")
    print(f"Общая масса: {stats['mass_sum']:.1f} кг")
    print(f"Средняя масса: {stats['average']:.1f} кг")
    print("По статусам:")
    for status, count in stats["by_status"].items():
        print(f"  {status}: {count}")
    if stats["heaviest"] is not None:
        heavy = stats["heaviest"]
        print(f"Самый тяжёлый груз: {heavy.number} ({heavy.mass:.1f} кг)")


def show_log(service):
    lines = service.repository.read_log()
    print("\n----- Журнал операций -----")
    if not lines:
        print("Журнал пуст.")
        return
    for line in lines[-12:]:
        print(line)


def print_menu():
    print()
    print("===== Система управления грузами =====")
    print("1. Создать груз")
    print("2. Показать все грузы")
    print("3. Изменить статус")
    print("4. Найти груз")
    print("5. Фильтрация")
    print("6. Статистика")
    print("7. Сохранить JSON")
    print("8. Загрузить JSON")
    print("9. Экспортировать CSV")
    print("10. Импортировать CSV")
    print("11. Журнал операций")
    print("0. Выход")
```

```
    print()


def main():
    repository = JsonRepository(
        json_path=os.path.join(DATA_DIR, "data.json"),
        csv_path=os.path.join(DATA_DIR, "data.csv"),
        backup_dir=os.path.join(DATA_DIR, "backup"),
        log_path=os.path.join(DATA_DIR, "operations.log"),
    )
    service = CargoService(repository)
    try:
        service.load()
        print(f"Загружено грузов из JSON: {len(service.cargos)}")
    except StorageError as exc:
        print(f"Ошибка хранилища: {exc}")
        print("Работа продолжается с пустым списком.")
        service.cargos = []

    while True:
        print_menu()
        choice = read_line("Выберите действие: ")
        try:
            if choice == "1":
                create_cargo(service)
            elif choice == "2":
                print_cargos(service.cargos, "Все грузы")
            elif choice == "3":
                change_status(service)
            elif choice == "4":
                search_cargo(service)
            elif choice == "5":
                filter_cargos(service)
            elif choice == "6":
                show_statistics(service)
            elif choice == "7":
                service.save()
                print(f"Данные сохранены: {repository.json_path}")
            elif choice == "8":
                service.load()
                print(f"Загружено записей: {len(service.cargos)}")
            elif choice == "9":
                service.export_csv()
                print(f"CSV создан: {repository.csv_path}")
            elif choice == "10":
                service.import_csv()
                print(f"Импортировано записей: {len(service.cargos)}")
            elif choice == "11":
                show_log(service)
            elif choice == "0":
                print("Работа программы завершена.")
                break
            else:
                print("Ошибка: выберите пункт меню от 0 до 11.")
        except CargoNotFoundError as exc:
            print(f"Исключение CargoNotFoundError: {exc}")
        except (ValidationError, StorageError, CargoError) as exc:
            print(f"Ошибка: {exc}")
        except EOFError:
            print("Ввод завершён.")
            break


if __name__ == "__main__":
    main()
```

Листинг — exceptions.py

```
"""Пользовательские исключения системы управления грузами."""


class CargoError(Exception):
    """Базовое исключение предметной области."""
```

```
class CargoNotFoundError(CargoError):
    """Попытка изменить или получить несуществующий груз."""


class ValidationError(CargoError):
    """Некорректные данные груза."""


class StorageError(CargoError):
    """Ошибка чтения или записи файла."""
```

Листинг — models.py

```
"""Модель груза."""

from exceptions import ValidationError

STATUSES = ("принят", "в пути", "доставлен", "отменён")


class Cargo:
    """Груз логистической службы."""

    def __init__(self, number, sender, recipient, mass, status="принят"):
        self.number = number
        self.sender = sender
        self.recipient = recipient
        self.mass = mass
        self.status = status

    @staticmethod
    def _check_text(value, field_name):
        if not isinstance(value, str) or not value.strip():
            raise ValidationError(f"поле «{field_name}» не должно быть пустым")
        return value.strip()

    @property
    def number(self):
        return self._number

    @number.setter
    def number(self, value):
        self._number = self._check_text(value, "номер")

    @property
    def sender(self):
        return self._sender

    @sender.setter
    def sender(self, value):
        self._sender = self._check_text(value, "отправитель")

    @property
    def recipient(self):
        return self._recipient

    @recipient.setter
    def recipient(self, value):
        self._recipient = self._check_text(value, "получатель")

    @property
    def mass(self):
        return self._mass

    @mass.setter
    def mass(self, value):
        if isinstance(value, bool) or not isinstance(value, (int, float)) or value <= 0:
            raise ValidationError("масса должна быть положительным числом")
        self._mass = float(value)
```

```
    @property
    def status(self):
        return self._status

    @status.setter
    def status(self, value):
        if value not in STATUSES:
            allowed = ", ".join(STATUSES)
            raise ValidationError(f"недопустимый статус. Доступны: {allowed}")
        self._status = value

    def to_dict(self):
        """Объект → словарь для JSON."""
        return {
            "number": self.number,
            "sender": self.sender,
            "recipient": self.recipient,
            "mass": self.mass,
            "status": self.status,
        }

    @classmethod
    def from_dict(cls, data):
        """Словарь → объект. classmethod обязателен по методичке."""
        return cls(
            number=data["number"],
            sender=data["sender"],
            recipient=data["recipient"],
            mass=data["mass"],
            status=data.get("status", "принят"),
        )

    def __str__(self):
        return (
            f"{self.number}: {self.sender} → {self.recipient}, "
            f"{self.mass:.1f} кг, {self.status}"
        )
```

Листинг — services.py

```
"""Бизнес-логика учёта грузов."""

from exceptions import CargoNotFoundError, ValidationError
from models import STATUSES, Cargo



class CargoService:
    """Операции над коллекцией грузов. Хранение передаётся снаружи."""

    def __init__(self, repository):
        self.repository = repository
        self.cargos = []

    def load(self):
        self.cargos = self.repository.load()
        self.repository.write_log(f"загрузка, записей: {len(self.cargos)}")

    def save(self):
        self.repository.save(self.cargos)
        self.repository.write_log(f"сохранение JSON, записей: {len(self.cargos)}")

    def export_csv(self):
        self.repository.export_csv(self.cargos)
        self.repository.write_log(f"экспорт CSV, записей: {len(self.cargos)}")

    def import_csv(self):
        self.cargos = self.repository.import_csv()
        self.repository.write_log(f"импорт CSV, записей: {len(self.cargos)}")

    def add(self, number, sender, recipient, mass, status="принят"):
        if self.find(number) is not None:
            raise ValidationError("груз с таким номером уже существует")
```

```
        cargo = Cargo(number, sender, recipient, mass, status)
        self.cargos.append(cargo)
        self.repository.write_log(f"создан груз {cargo.number}")
        return cargo

    def find(self, number):
        number_lower = number.lower()
        for cargo in self.cargos:
            if cargo.number.lower() == number_lower:
                return cargo
        return None

    def require(self, number):
        cargo = self.find(number)
        if cargo is None:
            raise CargoNotFoundError(f"груз «{number}» не найден")
        return cargo

    def change_status(self, number, status):
        cargo = self.require(number)
        if status not in STATUSES:
            raise ValidationError("недопустимый статус")
        if cargo.status == "отменён":
            raise ValidationError("нельзя изменить статус отменённого груза")
        if cargo.status == "доставлен" and status != "доставлен":
            raise ValidationError("доставленный груз нельзя вернуть в работу")
        cargo.status = status
        self.repository.write_log(f"статус {cargo.number} → {status}")
        return cargo

    def search(self, text):
        needle = text.lower()
        return [
            cargo
            for cargo in self.cargos
            if needle in cargo.number.lower()
            or needle in cargo.sender.lower()
            or needle in cargo.recipient.lower()
        ]

    def filter_by_status(self, status):
        return [cargo for cargo in self.cargos if cargo.status == status]

    def filter_by_mass(self, min_mass, max_mass):
        return [
            cargo
            for cargo in self.cargos
            if min_mass <= cargo.mass <= max_mass
        ]

    def statistics(self):
        total = len(self.cargos)
        mass_sum = 0
        by_status = {status: 0 for status in STATUSES}
        for cargo in self.cargos:
            mass_sum += cargo.mass
            by_status[cargo.status] += 1
        average = mass_sum / total if total else 0
        heaviest = None
        if self.cargos:
            heaviest = self.cargos[0]
            for cargo in self.cargos:
                if cargo.mass > heaviest.mass:
                    heaviest = cargo
        return {
            "count": total,
            "mass_sum": mass_sum,
            "average": average,
            "by_status": by_status,
            "heaviest": heaviest,
        }
```

Листинг — storage.py

```
"""Сохранение грузов в JSON и CSV. Слой хранения отделён от бизнес-логики."""
```

```
import csv
import json
import os
import shutil
from datetime import datetime

from exceptions import StorageError
from models import Cargo


class JsonRepository:
    """Репозиторий: загрузка и сохранение коллекции грузов."""

    def __init__(self, json_path, csv_path, backup_dir, log_path):
        self.json_path = json_path
        self.csv_path = csv_path
        self.backup_dir = backup_dir
        self.log_path = log_path

    def load(self):
        """Загружает грузы. Отсутствующий файл не считается ошибкой запуска."""
        if not os.path.exists(self.json_path):
            return []
        try:
            with open(self.json_path, "r", encoding="utf-8") as file:
                raw = json.load(file)
        except json.JSONDecodeError as exc:
            raise StorageError(f"JSON-файл повреждён: {exc}") from exc
        except OSError as exc:
            raise StorageError(f"не удалось прочитать файл: {exc}") from exc

        if not isinstance(raw, list):
            raise StorageError("JSON должен содержать список грузов")

        cargos = []
        for item in raw:
            cargos.append(Cargo.from_dict(item))
        return cargos

    def save(self, cargos):
        """Сохраняет грузы и делает резервную копию предыдущего файла."""
        folder = os.path.dirname(self.json_path)
        os.makedirs(folder, exist_ok=True)
        if os.path.exists(self.json_path):
            os.makedirs(self.backup_dir, exist_ok=True)
            stamp = datetime.now().strftime("%Y%m%d_%H%M%S")
            backup = os.path.join(self.backup_dir, f"data_{stamp}.json")
            shutil.copy2(self.json_path, backup)

        payload = [cargo.to_dict() for cargo in cargos]
        try:
            with open(self.json_path, "w", encoding="utf-8") as file:
                json.dump(payload, file, ensure_ascii=False, indent=2)
        except OSError as exc:
            raise StorageError(f"не удалось сохранить JSON: {exc}") from exc

    def export_csv(self, cargos):
        folder = os.path.dirname(self.csv_path)
        os.makedirs(folder, exist_ok=True)
        with open(self.csv_path, "w", newline="", encoding="utf-8") as file:
            writer = csv.DictWriter(
                file,
                fieldnames=["number", "sender", "recipient", "mass", "status"],
            )
            writer.writeheader()
            for cargo in cargos:
                writer.writerow(cargo.to_dict())

    def import_csv(self):
        if not os.path.exists(self.csv_path):
            raise StorageError("CSV-файл не найден")
        cargos = []
```

```
    with open(self.csv_path, "r", encoding="utf-8") as file:
        reader = csv.DictReader(file)
        for row in reader:
            cargos.append(
                Cargo.from_dict(
                    {
                        "number": row["number"],
                        "sender": row["sender"],
                        "recipient": row["recipient"],
                        "mass": float(row["mass"]),
                        "status": row["status"],
                    }
                )
            )
    return cargos

def write_log(self, message):
    folder = os.path.dirname(self.log_path)
    os.makedirs(folder, exist_ok=True)
    stamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with open(self.log_path, "a", encoding="utf-8") as file:
        file.write(f"{stamp} {message}\n")

def read_log(self):
    if not os.path.exists(self.log_path):
        return []
    with open(self.log_path, "r", encoding="utf-8") as file:
        return [line.rstrip("\n") for line in file]
```

6. Примеры работы программы

Ниже приведены снимки консольного приложения: меню, проверка ввода, создание грузов, поиск и статистика, пользовательское исключение и сохранение файлов.

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python4> python main.py

Загружено грузов из JSON: 0

===== Система управления грузами =====
1. Создать груз
2. Показать все грузы
3. Изменить статус
4. Найти груз
5. Фильтрация
6. Статистика
7. Сохранить JSON
8. Загрузить JSON
9. Экспортировать CSV
10. Импортировать CSV
11. Журнал операций
0. Выход

Выберите действие: 0
Работа программы завершена.
```

Рисунок 1 – Запуск и главное меню при пустом хранилище

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python4> python main.py

Загружено грузов из JSON: 0

===== Система управления грузами =====
1. Создать груз
2. Показать все грузы
3. Изменить статус
4. Найти груз
5. Фильтрация
6. Статистика
7. Сохранить JSON
8. Загрузить JSON
9. Экспортировать CSV
10. Импортировать CSV
11. Журнал операций
0. Выход

Выберите действие: 1
  Номер груза:
Ошибка: значение не должно быть пустым.
  Номер груза: абв
  Отправитель: G-20
  Получатель: Склад Север
  Масса, кг: Иванов
Ошибка: необходимо ввести число.
  Масса, кг: 0
Ошибка: масса должна быть больше нуля.
  Масса, кг: 12,5
Статус:
  1. принят
  2. в пути
  3. доставлен
  4. отменён
Выберите статус (1-4): 1
Груз создан.
абв: G-20 → Склад Север, 12.5 кг, принят
Операция создания завершена.

===== Система управления грузами =====
1. Создать груз
2. Показать все грузы
3. Изменить статус
4. Найти груз
5. Фильтрация
6. Статистика
7. Сохранить JSON
8. Загрузить JSON
```

Рисунок 2 – Проверка пустого номера, текста вместо массы и нулевой массы

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python4> python main.py

Загружено грузов из JSON: 0

===== Система управления грузами =====
1. Создать груз
2. Показать все грузы
3. Изменить статус
4. Найти груз
5. Фильтрация
6. Статистика
7. Сохранить JSON
8. Загрузить JSON
9. Экспортировать CSV
10. Импортировать CSV
11. Журнал операций
0. Выход

Выберите действие: 1
  Номер груза: G-21
  Отправитель: Озон
  Получатель: Петрова
  Масса, кг: 8
Статус:
  1. принят
  2. в пути
  3. доставлен
  4. отменён
Выберите статус (1-4): 2
Груз создан.
G-21: Озон → Петрова, 8.0 кг, в пути
Операция создания завершена.

===== Система управления грузами =====
1. Создать груз
2. Показать все грузы
3. Изменить статус
4. Найти груз
5. Фильтрация
6. Статистика
7. Сохранить JSON
8. Загрузить JSON
9. Экспортировать CSV
10. Импортировать CSV
11. Журнал операций
0. Выход

Выберите действие: 1
  Номер груза: G-22
  Отправитель: М.Видео
  Получатель: Сидоров
  Масса, кг: 30
Статус:
  1. принят
  2. в пути
  3. доставлен
  4. отменён
Выберите статус (1-4): 1
Груз создан.
G-22: М.Видео → Сидоров, 30.0 кг, принят
```

Рисунок 3 – Создание двух грузов, сохранение JSON и вывод списка

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python4> python main.py

Загружено грузов из JSON: 2

===== Система управления грузами =====
1. Создать груз
2. Показать все грузы
3. Изменить статус
4. Найти груз
5. Фильтрация
6. Статистика
7. Сохранить JSON
8. Загрузить JSON
9. Экспортировать CSV
10. Импортировать CSV
11. Журнал операций
0. Выход

Выберите действие: 6

----- Статистика -----
Количество грузов: 2
Общая масса: 38.0 кг
Средняя масса: 19.0 кг
По статусам:
  принят: 1
  в пути: 1
  доставлен: 0
  отменён: 0
Самый тяжёлый груз: G-22 (30.0 кг)

===== Система управления грузами =====
1. Создать груз
2. Показать все грузы
3. Изменить статус
4. Найти груз
5. Фильтрация
6. Статистика
7. Сохранить JSON
8. Загрузить JSON
9. Экспортировать CSV
10. Импортировать CSV
11. Журнал операций
0. Выход

Выберите действие: 4
Фрагмент номера, отправителя или получателя: Петрова

Поиск «Петрова»:
N   Номер       Отправитель       Получатель        Масса     Статус
1   G-21        Озон              Петрова           8.0       в пути
```

Рисунок 4 – Статистика, поиск по получателю и фильтр по статусу

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python4> python main.py

Загружено грузов из JSON: 2

===== Система управления грузами =====
1. Создать груз
2. Показать все грузы
3. Изменить статус
4. Найти груз
5. Фильтрация
6. Статистика
7. Сохранить JSON
8. Загрузить JSON
9. Экспортировать CSV
10. Импортировать CSV
11. Журнал операций
0. Выход

Выберите действие: 3
Номер груза: G-99
Статус:
  1. принят
  2. в пути
  3. доставлен
  4. отменён
Выберите статус (1-4): 1
Исключение CargoNotFoundError: груз «G-99» не найден

===== Система управления грузами =====
1. Создать груз
2. Показать все грузы
3. Изменить статус
4. Найти груз
5. Фильтрация
6. Статистика
7. Сохранить JSON
8. Загрузить JSON
9. Экспортировать CSV
10. Импортировать CSV
11. Журнал операций
0. Выход

Выберите действие: 3
Номер груза: G-21
Статус:
  1. принят
  2. в пути
  3. доставлен
```

Рисунок 5 – Исключение CargoNotFoundError и допустимая смена статуса

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python4> python main.py

Загружено грузов из JSON: 2

===== Система управления грузами =====
1. Создать груз
2. Показать все грузы
3. Изменить статус
4. Найти груз
5. Фильтрация
6. Статистика
7. Сохранить JSON
8. Загрузить JSON
9. Экспортировать CSV
10. Импортировать CSV
11. Журнал операций
0. Выход

Выберите действие: 7
Данные сохранены: C:\Users\Shmurdyakich\Desktop\pythonALL\python4\data\data.json

===== Система управления грузами =====
1. Создать груз
2. Показать все грузы
3. Изменить статус
4. Найти груз
5. Фильтрация
6. Статистика
7. Сохранить JSON
8. Загрузить JSON
9. Экспортировать CSV
10. Импортировать CSV
11. Журнал операций
0. Выход

Выберите действие: 9
CSV создан: C:\Users\Shmurdyakich\Desktop\pythonALL\python4\data\data.csv

===== Система управления грузами =====
1. Создать груз
2. Показать все грузы
3. Изменить статус
4. Найти груз
5. Фильтрация
6. Статистика
7. Сохранить JSON
8. Загрузить JSON
9. Экспортировать CSV
10. Импортировать CSV
11. Журнал операций
0. Выход

Выберите действие: 11

----- Журнал операций -----
2026-09-27 16:39:21 загрузка, записей: 0
2026-09-27 16:39:21 создан груз абв
2026-09-27 16:39:21 загрузка, записей: 0
2026-09-27 16:39:21 создан груз G-21
2026-09-27 16:39:21 создан груз G-22
```

Рисунок 6 – Повторное сохранение JSON, экспорт CSV и журнал операций

7. Ответы на контрольные вопросы

1. Что такое модуль Python?

Модуль — файл .py с функциями, классами и переменными. Его можно подключить из другого файла и тем самым разбить программу на части.

2. Чем import module отличается от from module import object?

import module даёт доступ через имя модуля: models.Cargo. from models import Cargo помещает имя Cargo сразу в текущую область видимости.

3. Что такое исключение?

Это объект, которым программа сообщает об ошибке. Пока его не обработают, нормальное выполнение прерывается.

4. Зачем нужны else и finally?

else выполняется только если в try не было исключения. finally выполняется в любом случае и подходит для завершения операции.

5. Как создать пользовательское исключение?

Нужно объявить класс, наследник Exception. В работе это CargoError и CargoNotFoundError.

6. Что произойдёт, если исключение не обработать?

Интерпретатор напечатает traceback и завершит программу. Поэтому в меню ошибки предметной области перехватываются.

7. Почему рекомендуется with open(...)?

Контекстный менеджер сам закрывает файл и при успехе, и при ошибке. Ресурс не остаётся открытым.

8. Чем json.dump отличается от json.dumps?

dump пишет JSON в файл, dumps возвращает строку. load читает файл, loads разбирает строку.

9. Что такое сериализация?

Это перевод объекта в формат, который можно сохранить или передать. Обратный перевод называется десериализацией.

10. Зачем в задании classmethod?

from_dict вызывается у класса, когда объекта ещё нет. Именно так словарь из JSON снова становится Cargo.

11. Почему хранение отделяют от бизнес-логики?

Сервис не знает, JSON это или CSV. Хранилище можно заменить, не переписывая правила статусов и статистику.

8. Вывод

В лабораторной работе №4 для варианта 20 разработана многофайловая система учёта грузов. Программа создаёт и ищет грузы, меняет статус, считает статистику, сохраняет состояние в JSON и выгружает CSV. Некорректный ввод и обращение к несуществующему грузу обрабатываются исключениями, а после повторного запуска ранее сохранённые данные загружаются автоматически.